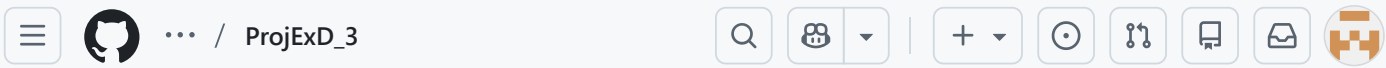
```python
import os
import random
import sys
import time
import pygame as pg


WIDTH = 1100  # ゲームウィンドウの幅
HEIGHT = 650  # ゲームウィンドウの高さ
NUM_OF_BOMBS = 5  # 爆弾の数
os.chdir(os.path.dirname(os.path.abspath(__file__)))


def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
    """
    オブジェクトが画面内or画面外を判定し，真理値タプルを返す関数
    引数：こうかとんや爆弾，ビームなどのRect
    戻り値：横方向，縦方向のはみ出し判定結果（画面内：True／画面外：False）
    """
    yoko, tate = True, True
    if obj_rct.left < 0 or WIDTH < obj_rct.right:
        yoko = False
    if obj_rct.top < 0 or HEIGHT < obj_rct.bottom:
        tate = False
    return yoko, tate


class Bird:
    """
    ゲームキャラクター（こうかとん）に関するクラス
    """
    delta = {  # 押下キーと移動量の辞書
        pg.K_UP: (0, -5),
        pg.K_DOWN: (0, +5),
        pg.K_LEFT: (-5, 0),
        pg.K_RIGHT: (+5, 0),
    }
    img0 = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
```

```python
        img = pg.transform.flip(img0, True, False)  # デフォルトのこうかとん（右向き）
        imgs = {  # 0度から反時計回りに定義
            (+5, 0): img,  # 右
            (+5, -5): pg.transform.rotozoom(img, 45, 0.9),  # 右上
            (0, -5): pg.transform.rotozoom(img, 90, 0.9),  # 上
            (-5, -5): pg.transform.rotozoom(img0, -45, 0.9),  # 左上
            (-5, 0): img0,  # 左
            (-5, +5): pg.transform.rotozoom(img0, 45, 0.9),  # 左下
            (0, +5): pg.transform.rotozoom(img, -90, 0.9),  # 下
            (+5, +5): pg.transform.rotozoom(img, -45, 0.9),  # 右下
        }

        def __init__(self, xy: tuple[int, int]):
            """
            こうかとん画像Surfaceを生成する
            引数 xy：こうかとん画像の初期位置座標タプル
            """
            self.img = __class__.imgs[(+5, 0)]
            self.rct: pg.Rect = self.img.get_rect()
            self.rct.center = xy

        def change_img(self, num: int, screen: pg.Surface):
            """
            こうかとん画像を切り替え，画面に転送する
            引数1 num：こうかとん画像ファイル名の番号
            引数2 screen：画面Surface
            """
            self.img = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
            screen.blit(self.img, self.rct)

        def update(self, key_lst: list[bool], screen: pg.Surface):
            """
            押下キーに応じてこうかとんを移動させる
            引数1 key_lst：押下キーの真理値リスト
            引数2 screen：画面Surface
            """
            sum_mv = [0, 0]
            for k, mv in __class__.delta.items():
                if key_lst[k]:
                    sum_mv[0] += mv[0]
                    sum_mv[1] += mv[1]
            self.rct.move_ip(sum_mv)
            if check_bound(self.rct) != (True, True):
                self.rct.move_ip(-sum_mv[0], -sum_mv[1])
            if not (sum_mv[0] == 0 and sum_mv[1] == 0):
                self.img = __class__.imgs[tuple(sum_mv)]
            screen.blit(self.img, self.rct)


    class Beam:
        """
        こうかとんが放つビームに関するクラス
        """
        def __init__(self, bird:"Bird"):
            """
            ビーム画像Surfaceを生成する
            引数 bird：ビームを放つこうかとん（Birdインスタンス）
            """
```

```python
            self.img = pg.image.load(f"fig/beam.png")
            self.rct = self.img.get_rect()
            self.rct.centery = bird.rct.centery  # ビームの中心縦座標 = こうかとんの中心縦座標
            self.rct.left = bird.rct.right  # ビームの左座標 = こうかとんの右座標
            self.vx, self.vy = +5, 0

        def update(self, screen: pg.Surface):
            """
            ビームを速度ベクトルself.vx, self.vyに基づき移動させる
            引数 screen：画面Surface
            """
            if check_bound(self.rct) == (True, True):
                self.rct.move_ip(self.vx, self.vy)
                screen.blit(self.img, self.rct)


    class Bomb:
        """
        爆弾に関するクラス
        """
        def __init__(self, color: tuple[int, int, int], rad: int):
            """
            引数に基づき爆弾円Surfaceを生成する
            引数1 color：爆弾円の色タプル
            引数2 rad：爆弾円の半径
            """
            self.img = pg.Surface((2*rad, 2*rad))
            pg.draw.circle(self.img, color, (rad, rad), rad)
            self.img.set_colorkey((0, 0, 0))
            self.rct = self.img.get_rect()
            self.rct.center = random.randint(0, WIDTH), random.randint(0, HEIGHT)
            self.vx, self.vy = +5, +5

        def update(self, screen: pg.Surface):
            """
            爆弾を速度ベクトルself.vx, self.vyに基づき移動させる
            引数 screen：画面Surface
            """
            yoko, tate = check_bound(self.rct)
            if not yoko:
                self.vx *= -1
            if not tate:
                self.vy *= -1
            self.rct.move_ip(self.vx, self.vy)
            screen.blit(self.img, self.rct)

    class Score:
        def __init__(self):
            self.fonto = pg.font.SysFont(None, 50)
            self.color = (0, 0, 255)
            self.score = 0
            self.image = self.fonto.render(f"Score: {self.score}", True, self.color)
            self.rct = self.image.get_rect()
            self.rct.center = (100, HEIGHT - 50)

        def update(self, screen: pg.Surface):
            self.image = self.fonto.render(f"Score: {self.score}", True, self.color)
            screen.blit(self.image, self.rct)
```

```python

def main():
    pg.display.set_caption("たたかえ！こうかとん")
    screen = pg.display.set_mode((WIDTH, HEIGHT))
    bg_img = pg.image.load("fig/pg_bg.jpg")
    bird = Bird((300, 200))
    # bomb = Bomb((255, 0, 0), 10)
    bombs = [Bomb((255, 0, 0), 10) for _ in range(NUM_OF_BOMBS)]
    # for _ in range(NUM_OF_BOMBS):
    #     bomb = Bomb((255, 0, 0), 10)
    #     bombs.append(bomb)
    score = Score()


    beam = None  # ゲーム初期化時にはビームは存在しない
    clock = pg.time.Clock()
    tmr = 0
    while True:
        for event in pg.event.get():
            if event.type == pg.QUIT:
                return
            if event.type == pg.KEYDOWN and event.key == pg.K_SPACE:
                beam = Beam(bird)

        # 1. 描画の準備（まず背景を塗る）
        screen.blit(bg_img, [0, 0])

        # 2. 衝突判定（スコア計算）
        for i, bomb in enumerate(bombs):
            # こうかとんと爆弾の衝突
            if bird.rct.colliderect(bomb.rct):
                fonto = pg.font.Font(None, 80)
                txt = fonto.render("Game Over", True, (255, 0, 0))
                screen.blit(txt, (WIDTH//2-150, HEIGHT//2))
                bird.change_img(8, screen)
                pg.display.update()
                time.sleep(1)
                return

            # ビームと爆弾の衝突
            if beam is not None:
                if beam.rct.colliderect(bomb.rct):
                    beam = None
                    bombs.pop(i)  # 爆弾を消す
                    score.score += 1  # ここでスコアを増やす！
                    bird.change_img(6, screen) # 喜びエフェクト（任意）
                    break # リスト操作中のループ抜け

        # 3. 各オブジェクトの移動と描画
        key_lst = pg.key.get_pressed()
        bird.update(key_lst, screen)

        if beam is not None:
            beam.update(screen)
            # 画面外に出たビームを消去（カクつき防止）
            if check_bound(beam.rct) != (True, True):
                beam = None
```

```python

            for bomb in bombs:
                bomb.update(screen)

            score.update(screen) # スコアの表示

            # 4. 最後に1回だけ画面を更新！
            pg.display.update()

            tmr += 1
            clock.tick(50)


    if __name__ == "__main__":
        pg.init()
        main()
        pg.quit()
        sys.exit()
```



Code Issues 2 Pull requests Agents Actions Projects More

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.

Branches

New branch

Overview Yours Active Stale All

Search branches...

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	22 minutes ago		Default		

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
issu1	7 minutes ago		0	1	
direction	49 minutes ago		0	0	
multibeam	1 hour ago		0	0	
score	1 hour ago		0	0	

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
issu1	7 minutes ago		0	1	
direction	49 minutes ago		0	0	
multibeam	1 hour ago		0	0	
score	1 hour ago		0	0	

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: issu1 ✓ Able to merge. These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests **Create pull request**

1 commit 1 file changed 1 contributor

Commits on Apr 28, 2026

改行削除 #1

c0a25062d3 committed 3 minutes ago ef7f3b1

Showing 1 changed file with **45 additions** and **102 deletions**. Split Unified

147 fight_kokaton.py

```
2   2   import random
3   3   import sys
4   4   import time
    5 + import math
5   6   import pygame as pg
6   7
7     - 
8   8   WIDTH = 1100  # ゲームウィンドウの幅
9   9   HEIGHT = 650  # ゲームウィンドウの高さ
10  10  NUM_OF_BOMBS = 5  # 爆弾の数
11  11  os.chdir(os.path.dirname(os.path.abspath(__file__)))
12  12
13    - 
14  13  def check_bound(obj_rct: pg.Rect) -> tuple[bool, bool]:
15    -     """
16    -     オブジェクトが画面内or画面外を判定し，真理値タプルを返す関数
17    -     引数：こうかとんや爆弾，ビームなどのRect
18    -     戻り値：横方向，縦方向のはみ出し判定結果（画面内：True／画面外：False）
19    -     """
20  14      yoko, tate = True, True
21  15      if obj_rct.left < 0 or WIDTH < obj_rct.right:
22  16          yoko = False
```

```
23  17      if obj_rct.top < 0 or HEIGHT < obj_rct.bottom:
24  18          tate = False
25  19      return yoko, tate
26  20
27      -
28  21  class Bird:
29      -     """
30      -     ゲームキャラクター（こうかとん）に関するクラス
31      -     """
32      -     delta = {  # 押下キーと移動量の辞書
    22  +     delta = {
33  23          pg.K_UP: (0, -5),
34  24          pg.K_DOWN: (0, +5),
35  25          pg.K_LEFT: (-5, 0),
36  26          pg.K_RIGHT: (+5, 0),
37  27      }
38  28      img0 = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
39      -     img = pg.transform.flip(img0, True, False)  # デフォルトのこうかとん（右向き）
40      -     imgs = {  # 0度から反時計回りに定義
41      -         (+5, 0): img,  # 右
42      -         (+5, -5): pg.transform.rotozoom(img, 45, 0.9),  # 右上
43      -         (0, -5): pg.transform.rotozoom(img, 90, 0.9),  # 上
44      -         (-5, -5): pg.transform.rotozoom(img0, -45, 0.9),  # 左上
45      -         (-5, 0): img0,  # 左
46      -         (-5, +5): pg.transform.rotozoom(img0, 45, 0.9),  # 左下
47      -         (0, +5): pg.transform.rotozoom(img, -90, 0.9),  # 下
48      -         (+5, +5): pg.transform.rotozoom(img, -45, 0.9),  # 右下
    29  +     img = pg.transform.flip(img0, True, False)
    30  +     imgs = {
    31  +         (+5, 0): img,
    32  +         (+5, -5): pg.transform.rotozoom(img, 45, 0.9),
    33  +         (0, -5): pg.transform.rotozoom(img, 90, 0.9),
    34  +         (-5, -5): pg.transform.rotozoom(img0, -45, 0.9),
    35  +         (-5, 0): img0,
    36  +         (-5, +5): pg.transform.rotozoom(img0, 45, 0.9),
    37  +         (0, +5): pg.transform.rotozoom(img, -90, 0.9),
    38  +         (+5, +5): pg.transform.rotozoom(img, -45, 0.9),
49  39      }
50  40
51  41      def __init__(self, xy: tuple[int, int]):
52      -         """
53      -         こうかとん画像Surfaceを生成する
54      -         引数 xy：こうかとん画像の初期位置座標タプル
55      -         """
56  42          self.img = __class__.imgs[(+5, 0)]
57  43          self.rct: pg.Rect = self.img.get_rect()
58  44          self.rct.center = xy
    45  +         self.dire = (+5, 0) # 演習4: 現在の向きを保持
59  46
60  47      def change_img(self, num: int, screen: pg.Surface):
61      -         """
62      -         こうかとん画像を切り替え，画面に転送する
63      -         引数1 num：こうかとん画像ファイル名の番号
64      -         引数2 screen：画面Surface
65      -         """
66  48          self.img = pg.transform.rotozoom(pg.image.load(f"fig/{num}.png"), 0, 0.9)
67  49          screen.blit(self.img, self.rct)
68  50
```

```
69  51      def update(self, key_lst: list[bool], screen: pg.Surface):
70          -       """
71          -       押下キーに応じてこうかとんを移動させる
72          -       引数1 key_lst：押下キーの真理値リスト
73          -       引数2 screen：画面Surface
74          -       """
75  52          sum_mv = [0, 0]
76  53          for k, mv in __class__.delta.items():
77  54              if key_lst[k]:
82  59                  self.rct.move_ip(-sum_mv[0], -sum_mv[1])
83  60          if not (sum_mv[0] == 0 and sum_mv[1] == 0):
84  61              self.img = __class__.imgs[tuple(sum_mv)]
        62      +           self.dire = tuple(sum_mv) # 演習4: 移動方向を更新
85  63          screen.blit(self.img, self.rct)
86  64
87          -
88  65  class Beam:
89          -   """
90          -   こうかとんが放つビームに関するクラス
91          -   """
92          -   def __init__(self, bird:"Bird"):
93          -       """
94          -       ビーム画像Surfaceを生成する
95          -       引数 bird：ビームを放つこうかとん（Birdインスタンス）
96          -       """
97          -       self.img = pg.image.load(f"fig/beam.png")
        66  +   def __init__(self, bird: Bird):
        67  +       self.img = pg.image.load("fig/beam.png")
        68  +       # 演習4: 向きに応じてビームを回転
        69  +       vx, vy = bird.dire
        70  +       angle = math.degrees(math.atan2(-vy, vx))
        71  +       self.img = pg.transform.rotozoom(self.img, angle, 1.0)
        72  +
98  73          self.rct = self.img.get_rect()
99          -       self.rct.centery = bird.rct.centery  # ビームの中心縦座標 = こうかとんの中心縦座
            標
100         -       self.rct.left = bird.rct.right  # ビームの左座標 = こうかとんの右座標
101         -       self.vx, self.vy = +5, 0
        74  +       self.rct.center = bird.rct.center # こうかとんの中心から発射
        75  +       self.vx, self.vy = vx, vy
102 76
103 77      def update(self, screen: pg.Surface):
104         -       """
105         -       ビームを速度ベクトルself.vx, self.vyに基づき移動させる
106         -       引数 screen：画面Surface
107         -       """
108         -       if check_bound(self.rct) == (True, True):
109         -           self.rct.move_ip(self.vx, self.vy)
110         -           screen.blit(self.img, self.rct)
111         -
        78  +       self.rct.move_ip(self.vx, self.vy)
        79  +       screen.blit(self.img, self.rct)
112 80
113 81  class Bomb:
114         -   """
115         -   爆弾に関するクラス
116         -   """
117 82      def __init__(self, color: tuple[int, int, int], rad: int):
```

```
118        -          """
119        -          引数に基づき爆弾円Surfaceを生成する
120        -          引数1 color：爆弾円の色タプル
121        -          引数2 rad：爆弾円の半径
122        -          """
123  83              self.img = pg.Surface((2*rad, 2*rad))
124  84              pg.draw.circle(self.img, color, (rad, rad), rad)
125  85              self.img.set_colorkey((0, 0, 0))
128  88              self.vx, self.vy = +5, +5
129  89
130  90          def update(self, screen: pg.Surface):
131        -          """
132        -          爆弾を速度ベクトルself.vx, self.vyに基づき移動させる
133        -          引数 screen：画面Surface
134        -          """
135  91              yoko, tate = check_bound(self.rct)
136  92              if not yoko:
137  93                  self.vx *= -1
153 109              self.image = self.fonto.render(f"Score: {self.score}", True, self.color)
154 110              screen.blit(self.image, self.rct)
155 111
156        -
157 112      def main():
158 113          pg.display.set_caption("たたかえ！こうかとん")
159 114          screen = pg.display.set_mode((WIDTH, HEIGHT))
160 115          bg_img = pg.image.load("fig/pg_bg.jpg")
161 116          bird = Bird((300, 200))
162        -      # bomb = Bomb((255, 0, 0), 10)
163 117          bombs = [Bomb((255, 0, 0), 10) for _ in range(NUM_OF_BOMBS)]
164        -      # for _ in range(NUM_OF_BOMBS):
165        -      #     bomb = Bomb((255, 0, 0), 10)
166        -      #     bombs.append(bomb)
167 118          score = Score()
    119    +     beams = []
168 120
169        -
170        -      beam = None  # ゲーム初期化時にはビームは存在しない
171 121          clock = pg.time.Clock()
172        -      tmr = 0
    122    +
173 123          while True:
174 124              for event in pg.event.get():
175 125                  if event.type == pg.QUIT:
176 126                      return
177 127                  if event.type == pg.KEYDOWN and event.key == pg.K_SPACE:
178        -                  beam = Beam(bird)
    128    +                  beams.append(Beam(bird))
179 129
180        -          # 1. 描画の準備（まず背景を塗る）
181 130              screen.blit(bg_img, [0, 0])
182        -
183        -          # 2. 衝突判定（スコア計算）
    131    +
    132    +          # 衝突判定
184 133              for i, bomb in enumerate(bombs):
185 134                  # こうかとんと爆弾の衝突
186 135                  if bird.rct.colliderect(bomb.rct):
193 142                      return
```

```
194 143
195 144                 # ビームと爆弾の衝突
196     -               if beam is not None:
    145 +               for j, beam in enumerate(beams):
197 146                     if beam.rct.colliderect(bomb.rct):
198     -                       beam = None
199     -                       bombs.pop(i)  # 爆弾を消す
200     -                       score.score += 1  # ここでスコアを増やす！
201     -                       bird.change_img(6, screen) # 喜びエフェクト（任意）
202     -                       break # リスト操作中のループ抜け
203     -
204     -           # 3. 各オブジェクトの移動と描画
205     -           key_lst = pg.key.get_pressed()
206     -           bird.update(key_lst, screen)
    147 +                       beams.pop(j)
    148 +                       bombs.pop(i)
    149 +                       score.score += 1
    150 +                       bird.change_img(6, screen)
    151 +                       break
    152 +
    153 +           # 画面外のビーム除去
    154 +           beams = [b for b in beams if check_bound(b.rct) == (True, True)]
207 155
208     -           if beam is not None:
    156 +           # 移動と描画
    157 +           for beam in beams:
209 158                 beam.update(screen)
210     -               # 画面外に出たビームを消去（カクつき防止）
211     -               if check_bound(beam.rct) != (True, True):
212     -                   beam = None
213     -
214 159           for bomb in bombs:
215 160               bomb.update(screen)
216 161
217     -           score.update(screen) # スコアの表示
218     -
219     -           # 4. 最後に1回だけ画面を更新！
220     -           pg.display.update()
    162 +           key_lst = pg.key.get_pressed()
    163 +           bird.update(key_lst, screen)
    164 +           score.update(screen)
221 165
222     -           tmr += 1
    166 +           pg.display.update()
223 167           clock.tick(50)
224 168
225     -
226 169   if __name__ == "__main__":
227 170       pg.init()
228 171       main()
```